

PROJET : AIRSENSE – STATION DE MESURE DE QUALITE DE L'AIR

1. Introduction et Objectifs

La problématique de notre projet était la suivante : « Durant l'hiver, il fait froid dehors, ce qui peut nous rendre réticents à aérer notre appartement, car cela fait entrer le froid et augmente donc la facture d'électricité ».

Nous avons donc choisi de réaliser le projet nommé AirSense. C'est une station portable qui analyse la qualité de l'air ambiant. Elle mesure la température, l'humidité, la présence de gaz et les particules fines. L'idée est de pouvoir surveiller la qualité de l'air dans un appartement grâce à un affichage en temps réel sur un écran OLED, afin d'aérer l'appartement lorsque cela est nécessaire.

2. Matériel utilisé

Pour réaliser ce projet, nous avons utilisé la carte microcontrôleur ESP8266 (NodeMCU) fournie, une carte programmable et possédant une connectivité Wifi. Nous avons utilisé un Shield Grove pour faciliter les branchements.

Voici la liste des composants connectés à notre système :

- **Capteur DHT11** : Pour mesurer la température et l'humidité (branché sur D2).
- **Capteur de Gaz (MQ)** : Pour détecter la présence de gaz volatils (branché sur A0).
- **Capteur de Particules (HM330X)** : Pour mesurer la pollution de l'air (PM1, PM2.5, PM10) via le bus I2C.
- **Écran OLED** : Pour afficher les résultats, branché aussi en I2C.
- **LED interne** : Utilisée comme témoin de fonctionnement (heartbeat).

3. Conception Logicielle

Nous avons respecté les principes de la programmation orientée objet demandés dans le sujet.

3.1 Architecture globale

Notre code est organisé autour d'une classe principale appelée Application. Cette classe gère l'initialisation de tous les composants et la boucle principale du programme (Voir Annexe 1).

Pour gérer les capteurs, nous avons créé une classe mère générique appelée Module. Cette classe définit ce que tous les objets connectés ont en commun, comme le numéro de pin et une méthode d'initialisation (Voir Annexe 1).

Ensuite, chaque composant spécifique (Led, Capteur de gaz, Écran, etc.) est une classe fille qui hérite de Module. Cela nous permet d'utiliser le polymorphisme (Voir Annexe 1).

3.2 Choix techniques et C++

Nous avons appliqué plusieurs concepts du C++ pour répondre aux cahiers des charges en termes d'utilisation du C++:

- **Héritage et Polymorphisme** : Toutes nos classes de capteurs héritent de Module. La méthode `init()` est virtuelle pure dans Module, ce qui oblige chaque capteur à définir sa propre façon de s'initialiser.
- **Encapsulation** : Les attributs (comme pin ou status) sont protégés ou privés. L'accès se fait via des méthodes publiques (getters).
- **Allocation dynamique** : Dans la classe Application, nous utilisons des pointeurs. Les capteurs sont créés avec `new` dans le `init()` et détruits avec `delete` dans le destructeur pour bien gérer la mémoire.
- **STL (Standard Template Library)** : Pour le capteur de particules `ParticulateSensor`, nous utilisons un `std::vector` pour gérer le buffer de réception des données I2C.
- **Surcharge d'opérateur** : Nous avons surchargé l'opérateur `bool` dans la classe Module pour pouvoir vérifier facilement l'état d'un capteur.

4. Fonctionnement de l'application

Le fonctionnement est décrit dans le fichier Application.cpp (Voir Annexe 2)

1. **Démarrage (setup)** : L'Arduino lance la méthode app.init().
2. **Initialisation** : L'application instancie tous les capteurs (LED, DHT, Gaz, Particules, Écran). Elle appelle la méthode init() de chacun pour lancer les communications (Wire pour l'I2C, Serial, etc.).
3. **Boucle principale (loop)** : La méthode app.run() est appelée en boucle.
 - La LED clignote pour montrer que le système est actif (toggle).
 - Chaque capteur lit ses données (read). Par exemple, le GasSensor calcule un ratio basé sur le voltage reçu.
 - L'écran récupère toutes ces valeurs et les affiche.
4. **Affichage** : L'écran OLED affiche la température, l'humidité et le taux de particules PM2.5. Nous avons ajouté une petite intelligence : l'écran affiche "GOOD", "FAIR" ou "BAD" selon le niveau de pollution, et "DETECTED!" si du gaz est repéré (Annexe 3)

5. Conclusion

Ce projet nous a permis de mettre en pratique la programmation orientée objet sur un système embarqué. Nous avons réussi à créer une station de qualité de l'air fonctionnelle pouvant être utilisée chez nous.

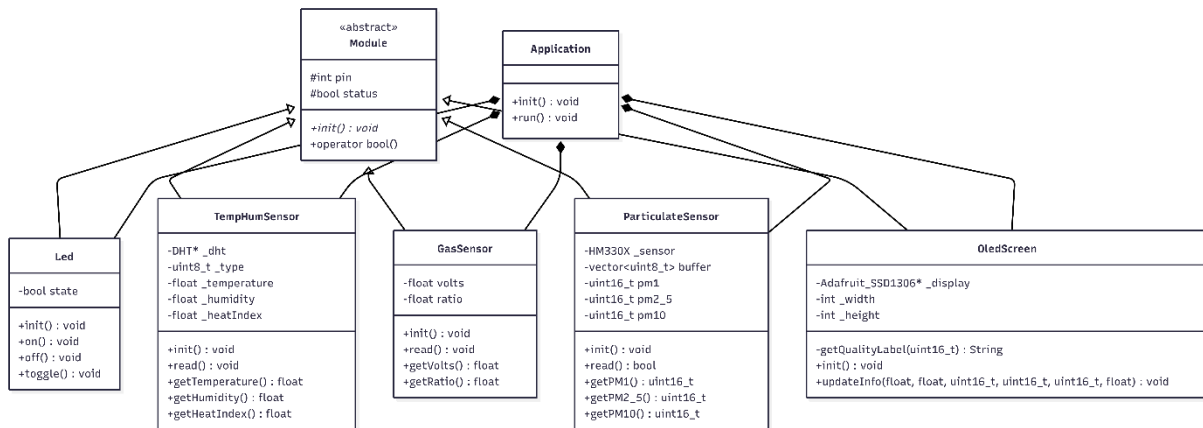
Objectifs atteints :

- Conception modulaire respectée (ajout facile d'un nouveau capteur en héritant de Module).
- Utilisation du C++ (Classes, Héritage, STL..).
- Le système lit correctement les valeurs et les affiche sur l'écran.

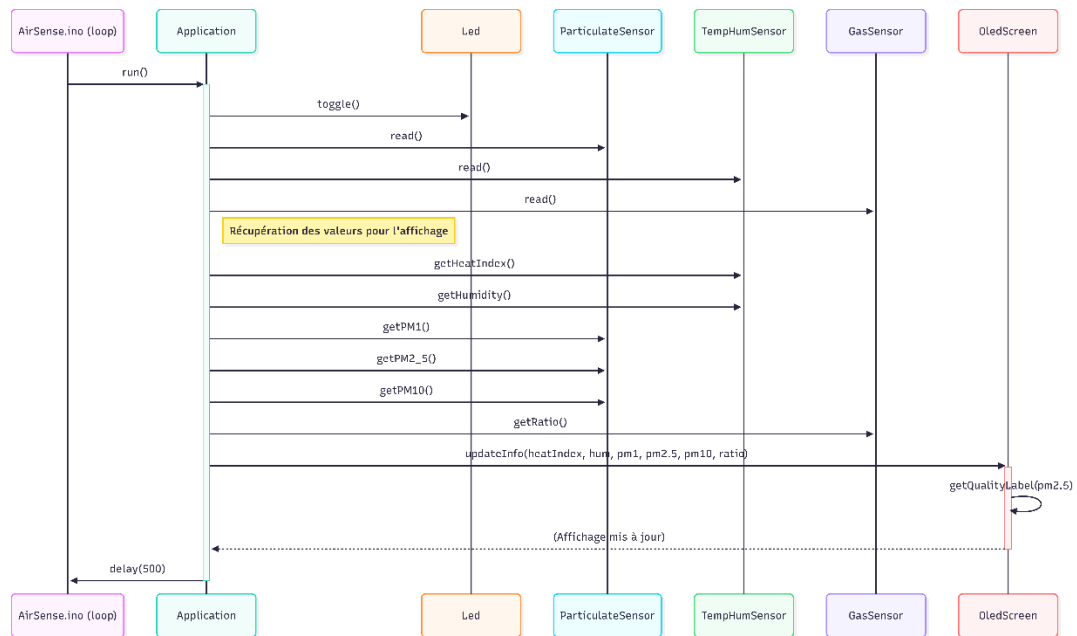
Difficultés rencontrées : La compréhension du système de classe héritages, la structure logicielle en plusieurs modules semblables, l'utilisation des exceptions (nous n'avons pas réussi les compiler avec l'Arduino IDE).

Améliorations possibles : Pour la suite, nous pourrions utiliser la connexion Wifi de l'ESP8266 pour envoyer ces données vers un serveur ou une application mobile. Cela permettrait de consulter la qualité de l'air à distance.

ANNEXE 1 – Diagramme de Classe UML



ANNEXE 2 – Diagramme de Séquence UML



ANNEXE 3 – Fonctionnement de l'écran Oled

